



1/1/2026

Rockfall

Submitted by:

Ancheta, Arem

Lardizabal, Timothy

Tan, Bryan

Submitted to:

Jayson Nacorda



Table of Contents

Building a Space Shooter

- Project Overview and Goals
 - Preparing the Development Environment
-

Part 1: Creating the Scene

- Creating the Unity Project
 - Saving and Managing Scenes
 - Importing and Verifying Assets
-

Part 2: Adding the Player Ship

- Creating the Ship Object
 - Adding and Aligning the Ship Model
 - Colliders and Movement Scripts
-

Part 3: Camera Follow System

- Camera Positioning

- Smooth Follow Behavior
-

Part 4: Adding the Space Station

- Station Setup
 - Collision Configuration
-

Part 5: Setting the Skybox

- Creating Skybox Materials
 - Applying Scene Lighting
-

Part 6: User Interface Canvas

- Canvas Configuration
 - UI Scaling and Resolution
-

Input and Flight Control

- Understanding Player Input
- Virtual Joystick Creation
- Input Manager System

- Ship Steering Logic
 - On-Screen Indicators
-

Adding Weapons and Targeting

- Creating Projectiles
 - Ship Weapon System
 - Fire Button UI
 - Target Reticle System
-

Asteroids, Damage, and Explosions

- Asteroid Creation
 - Asteroid Spawning System
 - Damage Handling
 - Explosion Effects
-

Audio, Menus, Death, and Polish

- Menu System Setup
- Game Manager Logic

- Boundaries and Warnings
- Visual and Audio Effects
- Final Game Polish

Introduction

Building a Space Shooter

Before you begin building Rockfall, it is important to understand the scope and structure of the game you are creating. Rockfall is a 3D space shooter where the player pilots a spacecraft, defends a space station, and survives waves of incoming asteroids. Each system in the game—movement, shooting, UI, audio, and effects—builds on the previous one.

This guide is structured to mirror how real games are developed: starting with a solid technical foundation, then layering gameplay systems, and finally adding polish. Think of this process as constructing a building. You cannot decorate the interior before the walls are stable, and you cannot add walls without a foundation.

By following this manual step by step, you will create not just a playable game, but a clean, organized Unity project that is easy to expand and maintain.

Game Concept Overview

The player controls a spacecraft tasked with defending a space station from falling asteroids, with the primary objective of destroying all incoming threats before they collide with the station. As the game progresses, the challenge increases through faster-moving asteroids, limited ammunition, and the need for precise ship movement and control. The player wins by surviving all asteroid waves while successfully keeping the space station intact.

Core Gameplay Loop

The player pilots a spacecraft using on-screen controls, aiming and shooting to destroy falling asteroids before they hit the station. Along the way, the player must avoid incoming debris, as collisions can damage both the ship and the station. By surviving each wave and clearing the area of threats, the player advances to the next level.

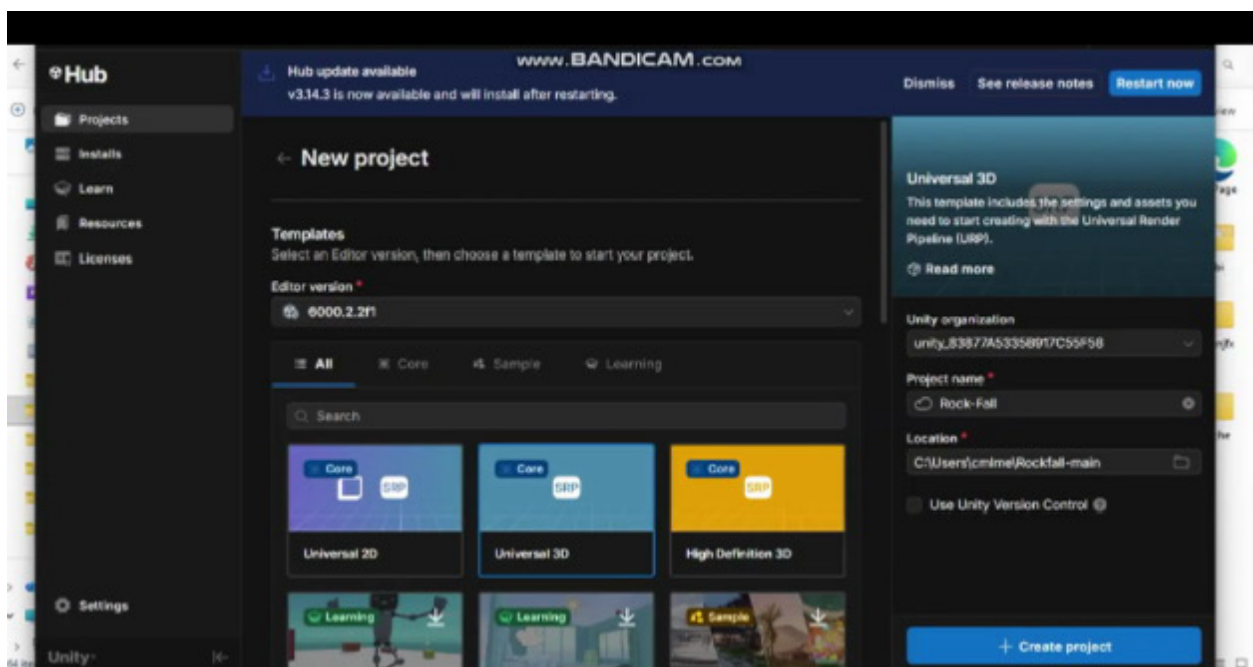
Part 1: Creating the Scene

This part focuses on setting up your Unity project correctly. A clean setup prevents errors later and ensures that assets, scripts, and scenes behave as expected.

Step 1: Create the Project

1. Open Unity Hub.
2. Click New Project.
3. Set the project name to Rockfall.
4. Choose 3D Core as the template.
 - 3D mode is required because the game uses 3D models, physics, and spatial movement.
5. Select a save location.
6. Click Create Project and wait for Unity to initialize.

Initialization may take several minutes, especially on the first run.



Step 2: Save the Scene

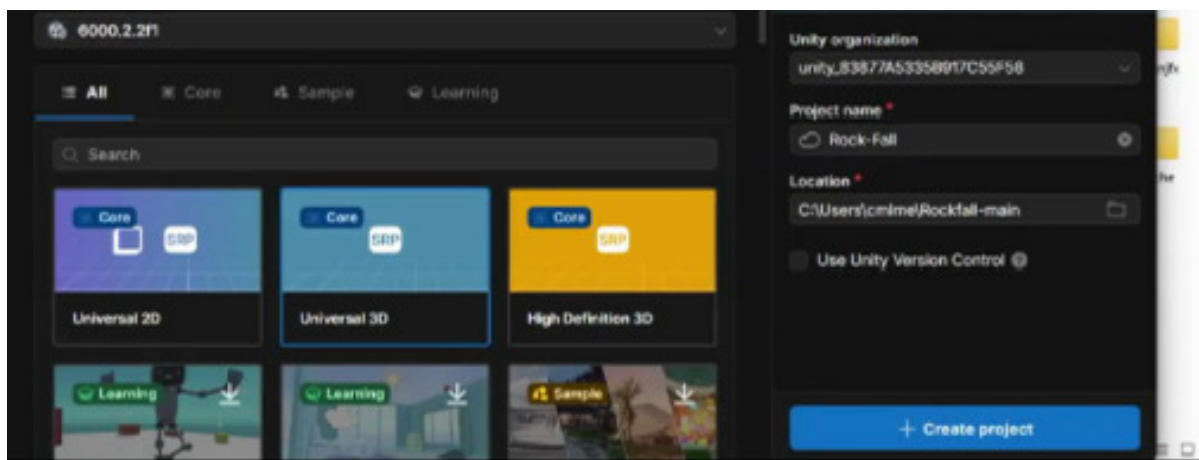
1. Open the File menu.
2. Select Save As.
3. Navigate to the Assets folder.
4. Name the scene Main.scene.
5. Click Save.

Saving early ensures your scene is registered correctly and prevents accidental data loss.

Step 3: Import the Assets

1. Locate the downloaded .unitypackage file.
2. Double-click it to open Unity's Import Package window.
3. Click Import All.
4. Wait for Unity to load all assets.

After importing, verify that folders such as Models, Scripts, Prefabs, and Skybox exist in the Assets panel. Proper asset organization is critical for large projects.



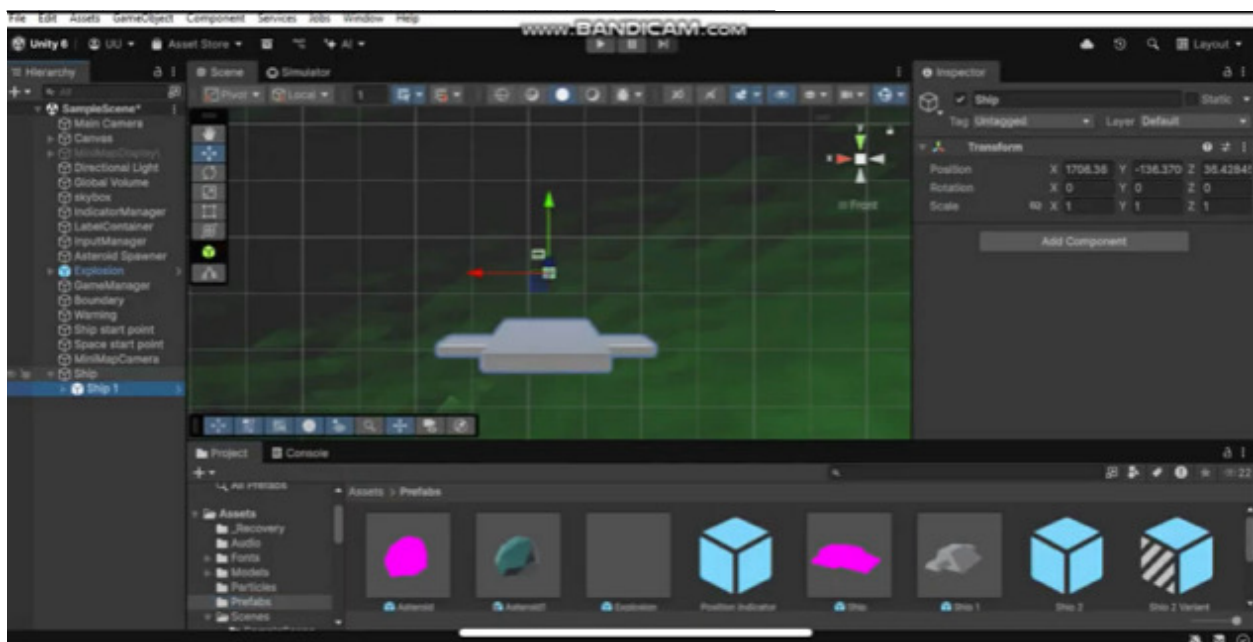
Part 2: Adding the Player Ship

The player ship is the core of gameplay. It must move smoothly, detect collisions, and support future systems such as weapons and input.

Step 1: Create the Ship Object

1. Go to GameObject → Create Empty.
2. Rename the object Ship.

Using an empty parent object allows scripts, colliders, and visuals to be managed cleanly.



Step 2: Add the Ship Model

1. Open the Models folder.
2. Drag the ship model onto the Ship object.
3. Rename the model Graphics.
4. Reset its position to (0, 0, 0).
5. Leave rotation at (-90, 0, 0) to align with Unity's coordinate system.

This rotation compensates for Blender's Z-up orientation versus Unity's Y-up system.



Step 3: Add Colliders and Movement

1. Select the Ship object.
2. Add a Box Collider.

- Enable Is Trigger.
 - Set size to approximately (2, 1.2, 3).
3. Add a new C# script called ShipThrust.cs.
 4. Configure the script to move the ship forward using Time.deltaTime.

This ensures consistent movement regardless of frame rate.



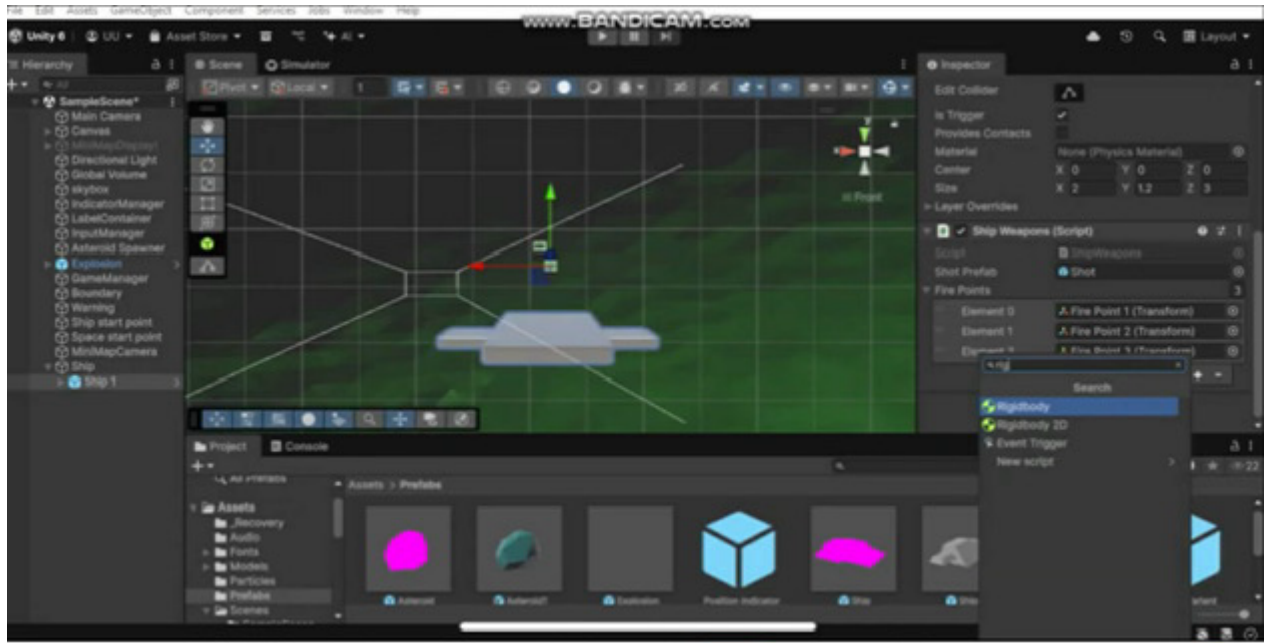
Part 3: Camera Follow

The camera must track the ship smoothly to create a cinematic and comfortable viewing experience.

Step 1: Create the Follow Script

1. Select Main Camera.

2. Add a new C# script named SmoothFollow.cs.
3. Assign the Ship as the target.



Step 2: Configure Camera Behavior

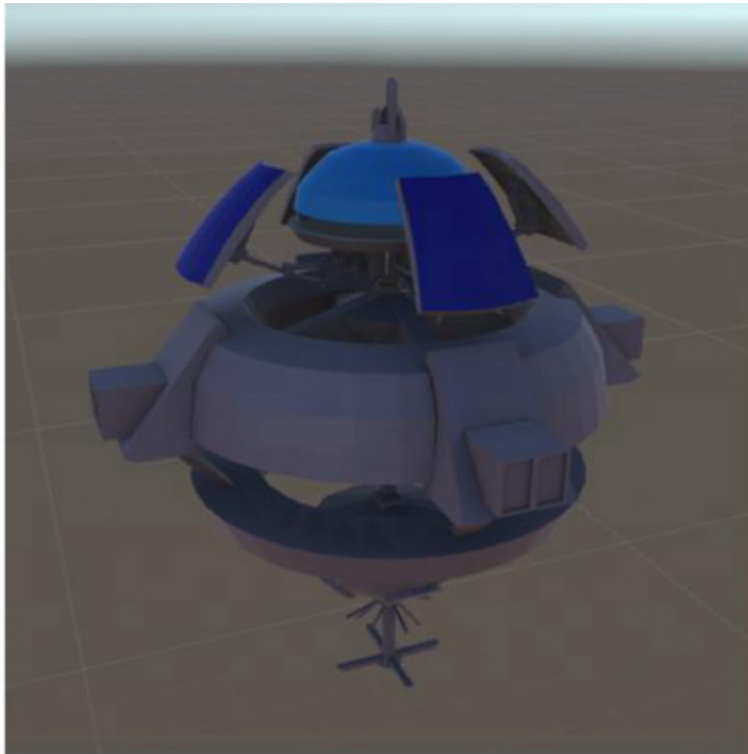
Adjust distance, height, and smoothing values until the ship remains visible and motion feels natural. Smooth camera lag enhances realism during turns.

Part 4: Adding the Space Station

The space station is the primary objective to protect.

1. Create an empty object named Space Station.
2. Drag the station model into it.
3. Reset its transform.

4. Ensure the model has a Mesh Collider for accurate collision detection.

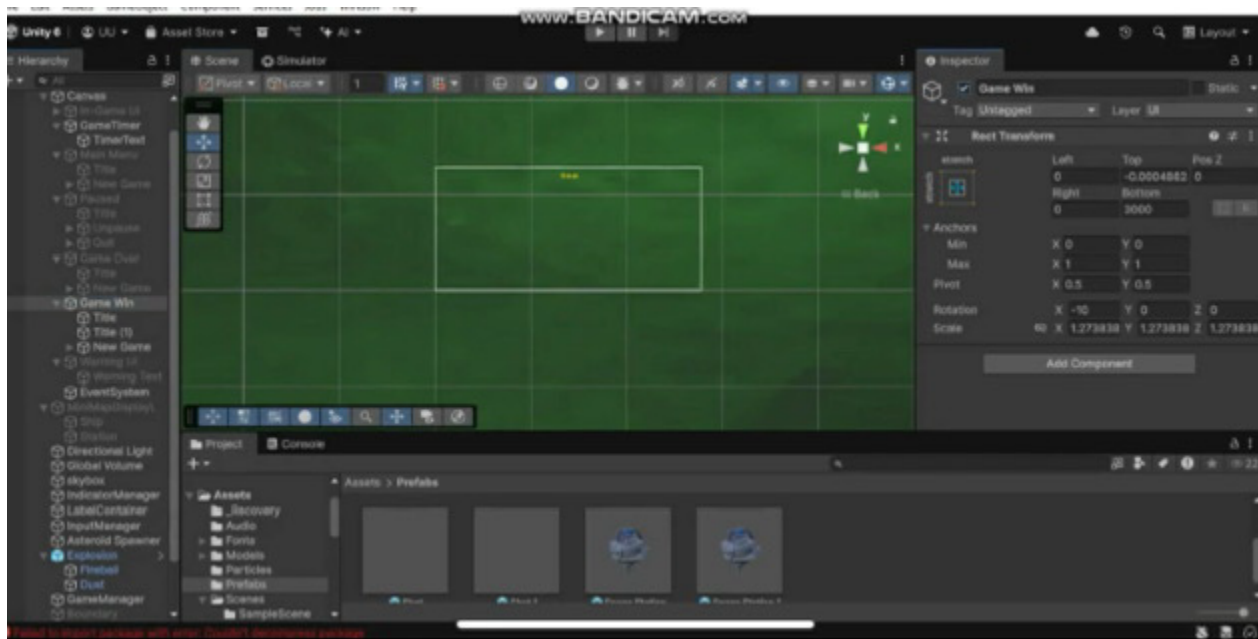


Part 5: Setting the Skybox

A skybox defines the background and lighting of the scene.

1. Create a new Material named Skybox.
2. Set Shader to Skybox / 6 Sided.
3. Assign textures to all six directions.
4. Apply the skybox via Window → Lighting → Settings.

This creates a believable space environment.



Part 6: User Interface Canvas

The UI handles player input, indicators, and menus.

1. Create a Canvas.
2. Set Render Mode to Screen Space – Camera.
3. Assign the Main Camera.
4. Enable Scale With Screen Size and set resolution to 1024×768 .

This ensures UI consistency across devices.



Input and Flight Control

Player input transforms user actions into ship movement.



Virtual Joystick Setup



You will create a joystick UI using panels and images, attach a VirtualJoystick.cs script, and track drag input. This joystick provides directional values used for steering.

Input Manager

An InputManager singleton centralizes input so any ship prefab can access joystick data, even when instantiated at runtime.

Ship Steering

The ShipSteering.cs script reads joystick input, rotates the ship smoothly, and gradually corrects roll to maintain orientation.

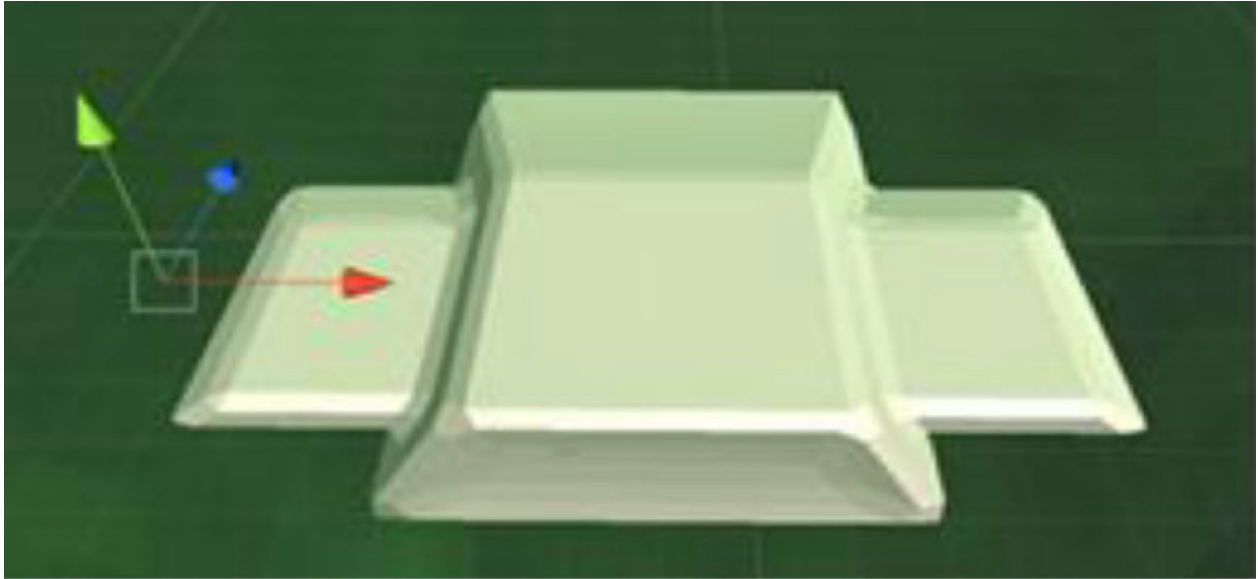


Adding Weapons and Targeting

Weapons introduce combat mechanics.

- Shots are created as prefabs with colliders and trail renderers
- Fire points alternate to simulate dual cannons
- A Fire Button UI triggers continuous firing
- A target reticle shows where shots will land

This creates responsive and readable combat.



Asteroids, Damage, and Explosions

Asteroids act as enemies and hazards.

- Asteroids are spawned dynamically via a spawner
- A damage system handles health and destruction
- Explosions use particle systems for visual feedback

Collisions now have consequences, increasing tension and challenge.

Audio, Menus, Death, and Polish

The final stage adds professionalism.

- Main Menu, Pause, and Game Over screens
- Game Manager controls flow and state
- Boundary warnings prevent leaving the play area
- Engine sounds, weapon audio, and explosion effects
- Visual polish like trails and dust particles

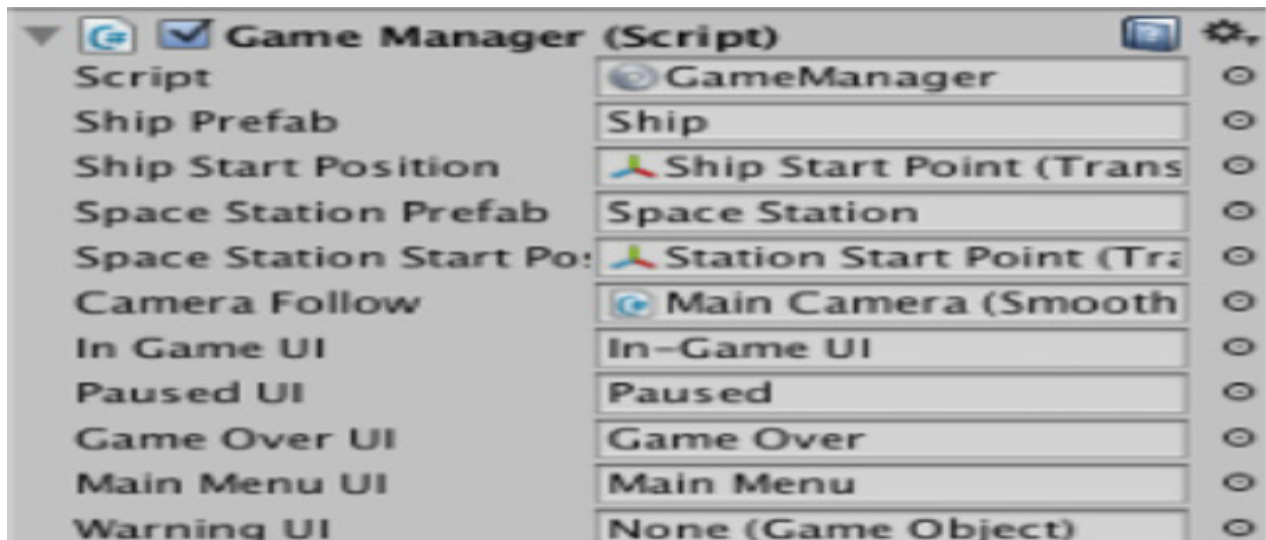


PAUSED

RESUME

GAME OVER

NEW GAME



Conclusion

By completing this manual, you have successfully created Rockfall, a fully functional 3D space shooter and that's no small achievement. Along the way, you've learned how different parts of a game player movement, input controls, UI design, combat mechanics, sound, and visual effects work together to create a seamless and engaging experience. You've seen firsthand how small changes in one system can affect the whole game, and you've learned to troubleshoot and refine your work until it feels right.

But Rockfall is more than just a finished project; it's a starting point. Now that you understand how a game comes together, you can take it further adding scoring systems, enemy variety, upgrades, new weapons, or even multiplayer modes. You've gained the tools to experiment, be creative, and make a game that reflects your own vision.

Beyond coding and mechanics, this project has given you a glimpse into the real-world process of game development: planning, iteration, problem-solving, and seeing your ideas come to life. Every bug you fixed, every mechanic you polished, and every decision you made has built not just a game, but your confidence as a developer.

Completing Rockfall shows that you can take a project from an idea on paper to a playable game, and that's a huge step toward creating even bigger and more ambitious games in the future. The skills you've gained here are the foundation your imagination and persistence will take it to the next level.